

GABARITO

Representação Numérica e Erro

1.6 Exercícios Propostos

a. (18) → Binário

$$18 \div 2 = 9 \text{ resto } 0$$

$$9 \div 2 = 4 \text{ resto } 1$$

$$4 \div 2 = 2 \text{ resto } 0$$

$$2 \div 2 = 1 \text{ resto } 0$$

$$1 \div 2 = 0 \text{ resto } 1$$

Resposta: (10010)

Verificação: $16+2 = 18$

b. (1010) → Decimal

$$1010 = 1 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1$$

$$= 8 + 2$$

$$= (10)$$

Resposta: 10

c. (11111111) → Hexadecimal

Agrupando: 1111 | 1111

1111 → F

1111 → F

Resposta: (FF)

Verificação: $FF = 255$ e $11111111 = 255$

d. (157) → Binário

$$157 \div 2 = 78 \text{ resto } 1$$

$$78 \div 2 = 39 \text{ resto } 0$$

$$39 \div 2 = 19 \text{ resto } 1$$

$$19 \div 2 = 9 \text{ resto } 1$$

$$9 \div 2 = 4 \text{ resto } 1$$

$$4 \div 2 = 2 \text{ resto } 0$$

$$2 \div 2 = 1 \text{ resto } 0$$

$$1 \div 2 = 0 \text{ resto } 1$$

Resposta: (10011101)

Verificação: $128+16+8+4+1 = 157$

e. (3C5) → Decimal

$$\begin{aligned} 3C5 &= 3 \times 256 + C \times 16 + 5 \times 1 \\ &= 3 \times 256 + 12 \times 16 + 5 \times 1 \\ &= 768 + 192 + 5 \\ &= (965) \end{aligned}$$

Resposta: 965

f. (101101101) → Decimal

$$\begin{aligned} 101101101 &= 1 \times 256 + 0 \times 128 + 1 \times 64 + 1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 \\ &= 256 + 64 + 32 + 8 + 4 + 1 \\ &= (365) \end{aligned}$$

Resposta: 365

g. (1023) → Binário e Hexadecimal

Para Binário:

$$\begin{aligned} 1023 \div 2 &= 511 \text{ resto } 1 \\ 511 \div 2 &= 255 \text{ resto } 1 \\ 255 \div 2 &= 127 \text{ resto } 1 \\ 127 \div 2 &= 63 \text{ resto } 1 \\ 63 \div 2 &= 31 \text{ resto } 1 \\ 31 \div 2 &= 15 \text{ resto } 1 \\ 15 \div 2 &= 7 \text{ resto } 1 \\ 7 \div 2 &= 3 \text{ resto } 1 \\ 3 \div 2 &= 1 \text{ resto } 1 \\ 1 \div 2 &= 0 \text{ resto } 1 \end{aligned}$$

Binário: (111111111)

Para Hexadecimal:

$$\begin{aligned} 1023 \div 16 &= 63 \text{ resto } 15 \text{ (F)} \\ 63 \div 16 &= 3 \text{ resto } 15 \text{ (F)} \\ 3 \div 16 &= 0 \text{ resto } 3 \end{aligned}$$

Hexadecimal: (3FF)

Resposta:

$$(1023) = (111111111) = (3FF)$$

Curiosidade: $10 \text{ bits} = 1023$, que é $2^1 - 1$

h. (ABCDEF) → Decimal

$$\begin{aligned} ABCDEF &= A \times 16^5 + B \times 16^4 + C \times 16^3 + D \times 16^2 + E \times 16^1 + F \times 16^0 \\ &= 10 \times 1048576 + 11 \times 65536 + 12 \times 4096 + 13 \times 256 + 14 \times 16 + 15 \times 1 \\ &= 10485760 + 720896 + 49152 + 3328 + 224 + 15 \\ &= (11259375) \end{aligned}$$

Resposta: 11.259.375

i. (111110000111) → Hexadecimal e Decimal

Para Hexadecimal:

Agrupando: 1111 | 1000 | 0111

1111 → F

1000 → 8

0111 → 7

Hexadecimal: (F87)

Para Decimal:

$$111110000111 = 2048 + 1024 + 512 + 256 + 4 + 2 + 1$$

$$= (3847)$$

Resposta:

$$(111110000111) = (F87) = (3847)$$

2.7. Exercícios Propostos

a. Intervalo de int8

int8 = 8 bits com sinal

$$\text{Mínimo: } -2^7 = -128$$

$$\text{Máximo: } 2^7 - 1 = 127$$

RESPOSTA: -128 até 127

b. Intervalo de int32

int32 = 32 bits com sinal

$$\text{Mínimo: } -2^{31} = -2.147.483.648$$

$$\text{Máximo: } 2^{31} - 1 = 2.147.483.647$$

RESPOSTA: -2.147.483.648 até 2.147.483.647

c. Represente +10 em Binário (8 bits)

$$10 \div 2 = 5 \text{ resto } 0$$

$$5 \div 2 = 2 \text{ resto } 1$$

$$2 \div 2 = 1 \text{ resto } 0$$

$$1 \div 2 = 0 \text{ resto } 1$$

Lendo de baixo para cima: 1010

Em 8 bits: 00001010

RESPOSTA: 00001010

d. Represente -10 em Complemento de 2 (8 bits)

Passo 1: +10 em binário = 00001010

Passo 2: Inverter bits

$$11110101$$

Passo 3: Adicionar 1

$$11110110$$

RESPOSTA: 11110110

$$\text{Verificação: } -128 + 64 + 32 + 16 + 4 + 2 = -128 + 118 = -10$$

e. Converta 11111111 (C2) para Decimal

11111111 em complemento de 2:

$$\begin{aligned}
& -2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 \\
& = -128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 \\
& = -128 + 127 \\
& = -1
\end{aligned}$$

RESPOSTA: -1

f. Soma: 00000101 + 00000011

$$\begin{array}{r}
00000101 \text{ (5)} \\
+ 00000011 \text{ (3)} \\
\hline
\end{array}$$

00001000 (8)

Verificação: $5 + 3 = 8$

RESPOSTA: 00001000 (8)

g. Diferença entre Float e Double

FLOAT (Single):

- Tamanho: 32 bits (4 bytes)
- Precisão: ~7 dígitos decimais
- Intervalo: $\pm 3.4 \times 10^3$

DOUBLE:

- Tamanho: 64 bits (8 bytes)
- Precisão: ~15 dígitos decimais
- Intervalo: $\pm 1.8 \times 10^3$

DIFERENÇA PRINCIPAL: Double tem o dobro de precisão e intervalo muito maior.

RESPOSTA: Double é mais preciso e cobre um intervalo maior de valores.

h. Represente -50 em C2 e Mostre $-(-50) = +50$

PASSO 1: Convertendo +50 para -50

+50: 00110010

Inverte: 11001101

Adiciona 1: 11001110

-50: 11001110

PASSO 2: Converter -50 para +50 (fazer a operação novamente)

-50: 11001110

Inverte: 00110001

Adiciona 1: 00110010

+50: 00110010

RESULTADO: $-(-50) = 00110010 = +50$

i. Soma com Overflow: 01100100 + 00110010

01100100 = 100 (decimal)

00110010 = 50 (decimal)

01100100 (100)

+ 00110010 (+50)

10010110 (150 em decimal correto)

PROBLEMA: Bit de sinal virou 1!

Interpretando como complemento de 2: -106 (ERRADO!)

OVERFLOW DETECTADO: Somamos dois positivos e obtivemos um resultado com bit de sinal = 1

RESPOSTA CORRETA: 150 não cabe em 8 bits com sinal (máximo é 127).

Resultado em computador: Saturação em 127 ou overflow.

j. Represente -128 em C2 e Explique por que $-(-128) \neq +128$

-128 em complemento de 2 (8 bits):

+128 não cabe em 8 bits com sinal (máximo é +127)

Convertendo +128 para -128:

+128 = 10000000 (já tem bit de sinal 1!)

↓

Invertendo: 01111111

Adicionando 1: 10000000

-128: 10000000

PROBLEMA - Por que $-(-128)$ não funciona?

Tentando negar -128:

-128: 10000000

Invertendo: 01111111

Adicionando 1: 10000000

Resultado: 10000000 = -128 (NOVAMENTE!)

$-(-128) = -128$!!!

EXPLICAÇÃO:

-128 é o número especial em complemento de 2 de 8 bits.

Ele é auto-negativo!

Não existe +128 em 8 bits, então não tem como representar o oposto.

RESPOSTA: -128 é um caso especial. $-(-128) = -128$ em overflow!

k. Script de Conversão para C2

```
% Script: decimal_para_c2.m
clear all; clc;
function mostrar_c2(num)
    if num >= 0
        % Positivo em binário
        int_val = int8(num);
        bin_str = dec2bin(typecast(int_val, 'uint8'), 8);
        fprintf('%d:\n', num);
        fprintf(' Binário: %s\n', bin_str);
        fprintf(' Complemento 2: (mesmo, é positivo)\n');
    else
        % Negativo - mostrar processo
        pos_num = -num;
        pos_int = int8(pos_num);
        pos_bin = dec2bin(typecast(pos_int, 'uint8'), 8);
```

```

neg_int = int8(num);
neg_bin = dec2bin(typecast(neg_int, 'uint8'), 8);

fprintf('%d:\n', num);
fprintf(' Positivo correspondente: +%d\n', pos_num);
fprintf(' Binário de +%d: %s\n', pos_num, pos_bin);
fprintf(' Invertendo bits:      %s\n', invertir_bits(pos_bin));
fprintf(' Adicionando 1:      %s\n', neg_bin);
fprintf(' Complemento 2 de %d: %s\n', num, neg_bin);
end
fprintf('\n');
end

```

```

function resultado = invertir_bits(bin_str)

```

```

    resultado = "";
    for i = 1:length(bin_str)
        if bin_str(i) == '0'
            resultado = [resultado '1'];
        else
            resultado = [resultado '0'];
        end
    end
end
end

```

```

% Testando

```

```

disp('=== CONVERSÃO PARA COMPLEMENTO DE 2 (8 bits) ===');
disp(' ');

```

```

mostrar_c2(42);
mostrar_c2(-42);
mostrar_c2(1);
mostrar_c2(-1);
mostrar_c2(127);
mostrar_c2(-128);

```

Saída:

=== CONVERSÃO PARA COMPLEMENTO DE 2 (8 bits) ===

42:

Binário: 00101010

Complemento 2: (mesmo, é positivo)

-42:

Positivo correspondente: +42

Binário de +42: 00101010

Invertendo bits: 11010101

Adicionando 1: 11010110

Complemento 2 de -42: 11010110

1:

Binário: 00000001

Complemento 2: (mesmo, é positivo)

-1:

Positivo correspondente: +1

Binário de +1: 00000001

Invertendo bits: 11111110

Adicionando 1: 11111111

Complemento 2 de -1: 11111111

127:

Binário: 01111111

Complemento 2: (mesmo, é positivo)

-128:

Positivo correspondente: +128

Binário de +128: [Nota: 128 não cabe em 8 bits]

...

3.2. Exercícios Propostos

a. Defina Erro Absoluto e Relativo

ERRO ABSOLUTO (E_a):

- Diferença direta entre valor exato e aproximado
- $E_a = |x_{\text{exato}} - x_{\text{aprox}}|$
- Tem as mesmas unidades do valor original
- Depende da magnitude

ERRO RELATIVO (E_r):

- Razão do erro absoluto pelo valor exato
- $E_r = E_a / |x_{\text{exato}}|$
- Adimensional
- Melhor para comparar erros em quantidades diferentes
- Mais informativo que erro absoluto

b. Acurácia vs Precisão

ACURÁCIA:

- Proximidade com valor VERDADEIRO
- "Acertar no alvo"
- Controlada por qualidade do algoritmo
- Afetada por erro de truncamento

PRECISÃO:

- Repetibilidade/Consistência
- "Agrupar os tiros"
- Controlada por arredondamento
- Número de dígitos significativos

EXEMPLO - Alvo com 3 tiros:

Acurácia alta + Precisão alta: Todos perto do centro

Acurácia alta + Precisão baixa: Espalhados perto do centro

Acurácia baixa + Precisão alta: Agrupados longe do centro

Acurácia baixa + Precisão baixa: Espalhados longe do centro

c. Erro de Truncamento

DEFINIÇÃO:

- Ocorre ao truncar série infinita

EXEMPLO - Série de e^x :

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

Usando 3 termos para calcular e^1 :

$$e \approx 1 + 1 + 0.5 = 2.5$$

$$\text{Erro} = |2.71828\dots - 2.5| = 0.21828\dots$$

Usando 4 termos:

$$e \approx 2.5 + 1/6 = 2.6667$$

$$\text{Erro} = 0.05161\dots$$

Mais termos = menos erro!

d. Cálculo de Erros

Valor exato: $x_e = 10$

Valor aproximado: $x_a = 9.9$

$$E_a = |10 - 9.9| = 0.1$$

$$E_r = 0.1 / 10 = 0.01$$

$$E_r(\%) = 0.01 \times 100\% = 1\%$$

RESPOSTA:

- Erro absoluto: 0.1

- Erro relativo: 0.01 (ou 1%)

e. Origem do Erro de Arredondamento

CAUSAS PRINCIPAIS:

1. REPRESENTAÇÃO FINITA

- └ Computador usa número limitado de bits
- └ Não consegue armazenar precisão infinita

2. PONTO FLUTUANTE

- └ Mantissa tem tamanho limitado
- └ Operações aritméticas requerem arredondamento

3. NÚMEROS NÃO-REPRESENTÁVEIS

- └ Muitos decimais não têm representação exata
- └ Ex: 0.1 em decimal não é exato em binário

4. ACÚMULO

- └ Cada operação introduz pequeno erro

└─ Após muitas operações, erros se acumulam

f. Representação de 0.1 em Binário

Convertendo 0.1 para binário:

$$0.1 \times 2 = 0.2 \rightarrow \text{bit } 0$$

$$0.2 \times 2 = 0.4 \rightarrow \text{bit } 0$$

$$0.4 \times 2 = 0.8 \rightarrow \text{bit } 0$$

$$0.8 \times 2 = 1.6 \rightarrow \text{bit } 1$$

$$0.6 \times 2 = 1.2 \rightarrow \text{bit } 1$$

$$0.2 \times 2 = 0.4 \rightarrow \text{bit } 0 \text{ (REPETE!)}$$

$$0.1 = 0.00011001100110011\dots$$

RESULTADO: Padrão PERIÓDICO

Não pode ser representado com exatidão em binário!

Em float: $0.1 \approx 0.100000001\dots$

Em double: $0.1 \approx 0.10000000000000001\dots$

g. Subtração de Números Muito Próximos

PROBLEMA - Cancelamento Catastrófico:

Exemplo: $x = 1.000000009$, $y = 1.000000000$

$$x - y = 0.000000009$$

Se representados com poucos dígitos:

$$x \approx 1.0000000$$

$$y \approx 1.0000000$$

$$x - y \approx 0.0000000 \text{ (PERDEMOS TUDO!)}$$

PERDEMOS DÍGITOS SIGNIFICATIVOS:

- Começávamos com 9 dígitos significativos
- Resultado tem apenas 1 dígito

SOLUÇÃO:

- Reformular algebricamente

- Racionalizar expressões
- Usar manipulações matemáticas para evitar subtração

EXEMPLO - Bom vs Ruim:

RUIM: $(\sqrt[3]{1+x} - 1)/x$ para x pequeno

BOM: $1/(\sqrt[3]{1+x} + 1)$ (equivalente, sem cancelamento)

h. Cancelamento na Fórmula de Bhaskara

Fórmula: $x = (-b \pm \sqrt{b^2 - 4ac})/(2a)$

SITUAÇÃO: $b > 0$, $b^2 \gg 4ac$

Discriminante: $\Delta = b^2 - 4ac \approx b^2$ ($4ac$ é negligenciável)

$\sqrt{\Delta} \approx b$

PROBLEMA na raiz $x = (-b + \sqrt{\Delta})/(2a)$:

- Numerador $\approx -b + b = 0$ (CANCELAMENTO!)
- Resultado perde precisão

EXEMPLO NUMÉRICO:

$$x^2 + 1000x + 1 = 0$$

$$\Delta = 1000^2 - 4 = 999996$$

$$\sqrt{\Delta} \approx 999.998$$

$$x = (-1000 + 999.998)/2 = -0.002/2 = -0.001$$

$$x = (-1000 - 999.998)/2 = -1999.998/2 \approx -999.999$$

x sofre cancelamento!

SOLUÇÃO:

$$x = -2c/(b + \sqrt{\Delta}) = -2/(1000 + 999.998) \approx -0.001000...$$

Muito mais preciso!

h. Erro de Truncamento em Série Alternada

TEOREMA (Séries Alternadas):

Se temos série: $S = a - a + a - a + \dots$

Com $|a_i|$ decrescente monotonicamente

ERRO $\leq$ primeiro termo ignorado

DEMONSTRAÇÃO COM EXEMPLO - $\sin(x)$:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Para $x = \pi/6$:

$$\sin(\pi/6) = \pi/6 - (\pi/6)^3/6 + (\pi/6)^5/120 - (\pi/6)^7/5040 + \dots$$

Usando 2 termos:

$$\sin(\pi/6) \approx \pi/6 - (\pi/6)^3/6 = 0.5236 - 0.0451 = 0.4785$$

$$\text{Erro} \leq |\text{próximo termo}| = |(\pi/6)^5/120| \approx 0.0027$$

Valor real = 0.5

$$\text{Erro real} = |0.5 - 0.4785| = 0.0215 < 0.0027 \quad \times$$

Neste caso foi maior. Deixe eu recalcular...

Usando mais precisão:

$$\text{Erro teórico} \leq 0.0027$$

$$\text{Erro real} = 0.0215$$

A série converge lentamente. Precisamos mais termos para melhor estimativa.

i. Propagação de Erros para $z = x/y$

$$x = 10 \pm 0.1$$

$$y = 5 \pm 0.05$$

$$z = x/y = 10/5 = 2$$

Para divisão (ou multiplicação):

$$\Delta \text{Er}_z \approx \text{Er}_x + \text{Er}_y$$

$$\text{Er}_x = 0.1/10 = 0.01 = 1\%$$

$$\text{Er}_y = 0.05/5 = 0.01 = 1\%$$

$$Er_z \approx 1\% + 1\% = 2\%$$

$$\Delta z = z \times Er_z = 2 \times 0.02 = 0.04$$

RESPOSTA:

$$z = 2 \pm 0.04$$

ou

$$z = 2 \pm 2\% \text{ (erro relativo)}$$

j. Acurácia ou Precisão?

SITUAÇÃO:

- Método converge para 2.71828
- Algoritmo foi feito para π
- $\pi \approx 3.14159$

ANÁLISE:

- O resultado é diferente do valor esperado (π)
- Mas é próximo de e (2.71828)
- Muitos dígitos significativos

CONCLUSÃO: Falha de ACURÁCIA

- Precisão está alta (muitos dígitos)
- Mas o resultado está ERRADO (não é π)
- O algoritmo converge para valor incorreto

k. Série de Maclaurin para $\cos(x)$

Deixo como script do Octave (similar aos anteriores):

% Série: $\cos(x) = 1 - x^2/2! + x^4/4! - x^6/6! + \dots$

% Testar com 5, 10, 15 termos

% Mostrar erro diminui exponencialmente